

Transação

Uma **transação** é uma unidade de trabalho atômica lógica que contém uma ou mais instruções SQL. Uma transação agrupa as instruções SQL para que elas sejam todas **confirmadas**, o que significa que elas são **aplicadas** ao banco de dados ou todas **revertidas**, o que significa que elas são **desfeitas** do banco de dados. O Database atribui a cada transação um identificador exclusivo chamado **ID de transação (transaction ID)**.

Todas as transações da Oracle cumprem as propriedades básicas de uma transação conforme as **propriedades ACID**. ACID é um acrônimo para o seguinte:

- Atomicidade

Todas as tarefas de uma transação são executadas ou nenhuma delas será executada. Não há transações parciais. Por exemplo, se uma transação começar a atualizar 100 linhas, mas o sistema falhar após 20 atualizações, o banco de dados reverterá as alterações nessas 20 linhas.

- Consistência

A transação leva o banco de dados de um estado consistente para outro estado consistente. Por exemplo, em uma transação bancária que debita de uma conta poupança e credita em uma conta corrente, uma falha não deve fazer com que o banco de dados credite apenas uma conta, o que levaria a dados inconsistentes.

- Isolamento

O efeito de uma transação não é visível para outras transações até que a transação seja confirmada. Por exemplo, um usuário que atualiza a tabela hr.employees não vê as alterações não confirmadas que são feitas na tabela hr.employees simultaneamente por outro usuário.

- Durabilidade

Alterações feitas por transações confirmadas são permanentes. Após a conclusão de uma transação, o banco de dados garante, por meio de seus mecanismos de recuperação, que as alterações da transação não serão perdidas.

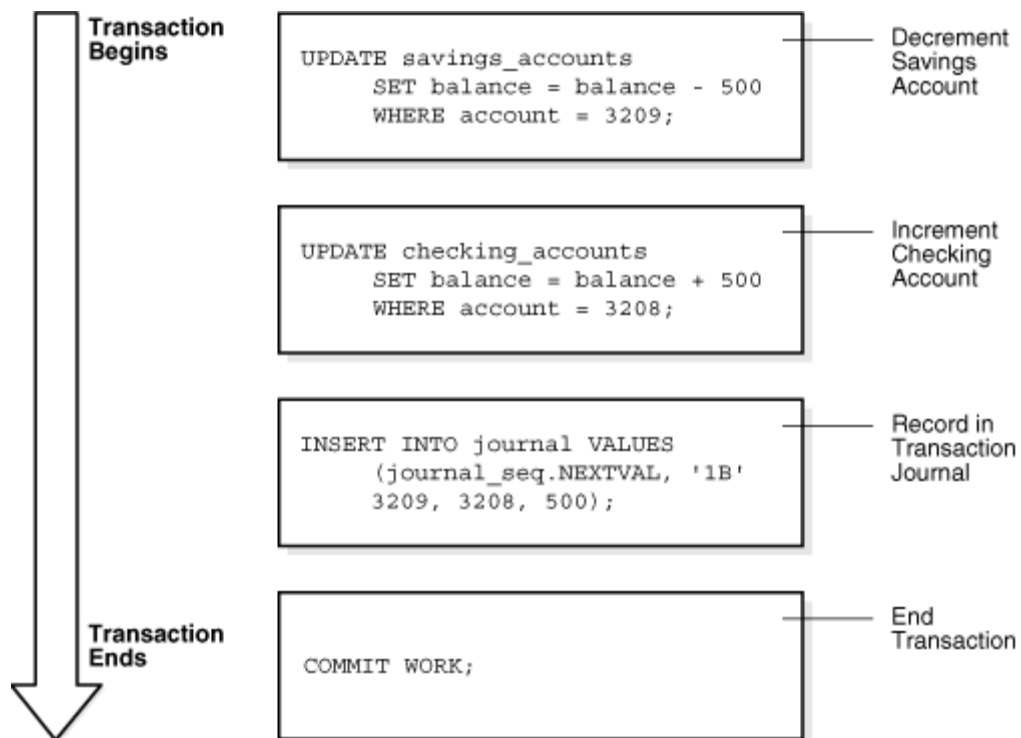
O uso de transações é uma das maneiras mais importantes em que um sistema de gerenciamento de banco de dados se difere de um sistema de gerenciamento de arquivos.

Exemplo de transação: débito e crédito da conta

Para ilustrar o conceito de uma transação, considere uma base de dados bancária. Quando um cliente transfere dinheiro de uma conta poupança para uma conta corrente, a transação deve consistir em três operações distintas:

- Diminuir o saldo da conta de poupança
- Aumentar o saldo de uma conta corrente
- Registrar a operação em um diário

Exemplo.



Estrutura de uma transação

Uma transação de banco de dados consiste em uma ou mais declarações. Especificamente, uma transação consiste em uma das seguintes instruções:

- Uma ou mais instruções de linguagem de manipulação de dados (DML) que em conjunto constituem uma alteração atômica para o banco de dados.
- Uma declaração de linguagem de definição de dados (DDL)

Uma transação é iniciada quando a primeira instrução SQL é executada. Uma **instrução SQL executável** é uma instrução SQL que gera chamadas para uma instância de banco de dados, incluindo instruções DML e DDL.

O exemplo a seguir executa uma instrução UPDATE para iniciar uma transação e consulta V\$TRANSACTION para obter detalhes sobre a transação:

```
UPDATE hr.employees  
SET salary=salary;
```

```
SELECT XID AS "txn id", XIDUSN AS "undo seg", XIDSLOT AS "slot",  
XIDSQN AS "se", STATUS AS "txn status"  
FROM V$TRANSACTION;
```

Uma transação é encerrada quando ocorre qualquer uma das seguintes ações:

- Um usuário emite uma declaração COMMIT ou ROLLBACK sem uma cláusula SAVEPOINT.
- Um usuário executa um comando DDL como CREATE, DROP, RENAME ou ALTER.
- Um usuário sai normalmente da maioria dos utilitários e ferramentas, fazendo com que a transação atual seja implicitamente confirmada. Esse comportamento pode ser configurado na aplicação
- Um processo cliente encerra de maneira anormal, fazendo com que a transação seja implicitamente revertida.

EXEMPLO PRÁTICO

T0

```
COMMIT;
```

Esta instrução termina qualquer transação existente na sessão.

T1

```
SET TRANSACTION NAME 'sal_update';
```

Esta instrução inicia uma transação e nomeia `sal_update`.

T2

```
UPDATE employees  
  SET salary = 7000  
  WHERE last_name = 'Banda';
```

Esta declaração atualiza o salário do funcionário Banda para 7000.

T3

```
SAVEPOINT after_banda_sal;
```

Esta declaração cria um savepoint chamado `after_banda_sal`

T4

```
UPDATE employees  
  SET salary = 12000  
  WHERE last_name = 'Greene';
```

Esta declaração atualiza o salário de Greene para 12000.

T5

```
SAVEPOINT after_greene_sal;
```

Esta declaração cria um savepoint denominado `after_greene_sal`

T6

```
ROLLBACK TO SAVEPOINT  
  after_banda_sal;
```

Essa declaração reverte a transação para t3, desfazendo a atualização para o salário de Greene em t4. A transação `sal_update` não terminou.

T7

```
UPDATE employees  
  SET salary = 11000  
  WHERE last_name = 'Greene';
```

Esta declaração atualiza o salário para Greene para 11000 na transação `sal_update`

T8

```
ROLLBACK;
```

Essa instrução reverte todas as alterações na transação `sal_update`, finalizando a transação.

PERGUNTA.

Nesse momento, qual é o salário do funcionário de sobrenome 'Banda'?

Nesse momento, qual é o salário do funcionário de sobrenome 'Greene'?

T9

```
SET TRANSACTION NAME 'sal_update2';
```

Esta declaração inicia uma nova transação na sessão e nomeia-a `sal_update2`.

T10

```
UPDATE employees  
  SET salary = 7050  
  WHERE last_name = 'Banda';
```

Esta declaração atualiza o salário do funcionário Banda para 7050.

T11

```
UPDATE employees  
  SET salary = 10950  
  WHERE last_name = 'Greene';
```

Esta declaração atualiza o salário de Greene para 10950.

T12

```
COMMIT;
```

Esta declaração confirma todas as alterações feitas na transação `sal_update2`, finalizando a transação.